Express.js — Day 13

JSON / Body Parsing + Practical API Structure • Long-Term Reference Notes

Day 13 Goal

Understand how Express receives JSON request bodies, how **express.json()** makes parsed data available through **req.body**, and how validation middleware checks incoming data before a route handler runs.

1. Receiving JSON

```
POST /users
Content-Type: application/json

{
"name": "Bhavana",
"age": 26
}
```

The client sends JSON in the request body. Express needs JSON parsing middleware before the route can conveniently use that data.

2. express.json()

```
app.use(express.json());
```

express.json() processes incoming JSON request bodies so the parsed data becomes available through **req.body**.

```
const express = require("express");
const app = express();

app.use(express.json());

app.post("/users", (req, res) => {
console.log(req.body);
res.json(req.body);
});
```

3. Request Flow

```
Client
↓
POST /users
↓
JSON request body
↓
express.json()
↓
req.body
↓
Route handler
```

Express processes middleware in registration order, so the JSON parser is registered before routes that need **req.body**.

4. Postman Experiment

A POST request to `http://localhost:3000/users` was sent with raw JSON containing `name: Bhavana` and `age: 26`. The response returned the same JSON and the terminal showed the parsed

JavaScript object. This confirmed that **req.body** contained the parsed data.

5. Validation Middleware

```
const validateUser = (req, res, next) => {
const { name, age } = req.body;

if (!name || age < 0) {
return res.status(400).send("Invalid user data");
}

next();
};
```

The middleware checks that **name** exists and **age** is not negative. Valid data continues with **next()**; invalid data receives a 400 response and stops.

6. Using Validation on a Route

```
app.post("/users", validateUser, (req, res) => {
console.log(req.body);
res.status(201).json(req.body);
});
```

The route runs **validateUser** first. Only if validation calls **next()** does the main route handler execute.

7. Valid Request Flow

```
POST /users
↓
express.json()
↓
req.body
↓
validateUser
↓
valid data
↓
next()
↓
route handler
↓
201 Created
```

The valid request with age 26 returned **201 Created** and the JSON user object.

8. Invalid Request Flow

```
POST /users
↓
express.json()
↓
req.body
↓
validateUser
↓
invalid data
↓
400 Bad Request
↓
STOP
```

The invalid request with age -50 returned **400 Bad Request** and **Invalid user data**.

9. Important Bug We Discovered

The first validation version sent the 400 response but then still executed **next()**:

```
if (!name || age < 0) {
res.status(400).send("Invalid user data");
}

next();
```

> This caused **ERR_HTTP_HEADERS_SENT: Cannot set headers after they are sent to the client**. The middleware sent a 400 response, then **next()** allowed the route handler to run and attempt another response.

10. Why return Matters

```
if (!name || age < 0) {
return res.status(400).send("Invalid user data");
}

next();
```

The **return** exits the middleware immediately after sending the response. Therefore **next()** does not execute for invalid data.

```
Invalid → send 400 → return → STOP
Valid → next() → route handler
```

11. Connection to Day 11

> This is the same middleware principle from Day 11: **allow** → **next()** or **reject** → **response and stop**. Day 13 applied it to request-body validation.

12. Status Codes Used

Status	Meaning	Use
201	Created	Valid user data accepted by the POST route.
400	Bad Request	Invalid user data rejected.

13. Day 13 Assessment — Reference Answers

1. express.json() processes incoming JSON request bodies so parsed data becomes available through **req.body**.

2. The parsed request data is available through **req.body**.

3. Validation middleware checks incoming data before the route handler. Valid data calls **next()**; invalid data sends a response and stops.

4. The **return** prevents **next()** from executing after the 400 response has already been sent.

14. Long-Term Mental Model

```
Incoming JSON
↓
express.json()
↓
req.body
↓
Validation middleware
↓
Valid? — NO → 400 response + STOP
|
YES
↓
next()
↓
Route handler
↓
201 Created
```

Core takeaway: `express.json()` parses JSON into `req.body`; validation protects the route from invalid input; and once a response is sent, the request must not continue to another response.

Day 13 Completion

Express.js Day 13 — COMPLETE ■
You implemented JSON body parsing, validation middleware, tested valid and invalid requests in Postman, and debugged the `ERR_HTTP_HEADERS_SENT` issue.